



Web Programming

Spring 2026

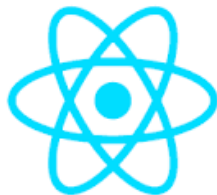
#12

Chi-Jen Wu



Topics

- The history and concepts of World Wide Web
- Web data protocols
 - HTTP
 - HTML, CSS
- Web JavaScript programming
- Cookies
- **Web Frontend frameworks**
- Web Backend frameworks
- RESTful API design



React



functional component state 狀態

```
<script>
const { useState, useEffect } = React; ←

function Welcome() { ←
  const [name, setName] = useState("CGU");
  return <h1>Hello, {name}</h1>;
}
ReactDOM.render(
  <Welcome />, ←
  document.getElementById('root')
);
</script>
```

初始值

functional component state 狀態

- `const { useState, useEffect } = React;`
 - `useState`
 - 一個兩個值的 array
 - 第一個值是 state
 - 第二個值是用來更新 state 的函式
 - 當 **state** 值改變，就會觸發 **re-render**
 - `useEffect`
 - 任何會產生 Side Effect 的行為 (改變state 狀態)，就會呼叫，等等會出現
 - `const [name, setName] = useState("CGU");`
 - name  狀態值
 - State
 - setName
 - Callback function  改變狀態值的 function



時鐘例子：state 寫法

```
<script>
const { useState, useEffect } = React; ←
function Clock() {
  const [mydate, setmydate] = useState(new Date()); ←
  ← useEffect(() => {
    setInterval(() => {
      setmydate(mydate => new Date()); ←
    }, 1000);
  }, []);

  return (
    <div>
      <h1>長庚標準時間 # state</h1>
      <h2>目前為： {mydate.toLocaleTimeString()}</h2>
    </div>
  );
}
↓
ReactDOM.render(<Clock />, document.getElementById('root'));
</script>
```



HTML

```
1 <div id="root">
2 </div>
```

CSS

JS (Babel)

```
28 const { useState, useEffect } = React;
29 function Clock() {
30   const [mydate, setmydate] = useState(new Date());
31   useEffect(() => {
32     setInterval(() => {
33       setmydate(mydate => new Date());
34       // mydate = new Date();
35     }, 1000);
36   });
37
38   return (
39     <div>
40       <h1>長庚標準時間 # state</h1>
41       <h2>目前為： {mydate.toLocaleTimeString()}</h2>
42     </div>
43   );
44 }
45 ReactDOM.render(<Clock />, document.getElementById('root'));
```

長庚標準時間 # state

目前為： 上午12:00:14

functional component state 狀態

- `const { useState, useEffect } = React;`
 - `useState`
 - state和更新 state 的函式
 - 當 **state** 值改變，就會觸發 **re-render**
 - `useEffect`
 - 任何會產生 Side Effect 的行為 (改變state 狀態，render 後，就會呼叫)
 - **React Hook**

React Hook : **useEffect** 改變 state 狀態

- 兩個參數
 - 第一個參數是 Effect function
 - 第二個則是 dependency array
 - [name, mydate]

```
useEffect(() => {  
  // setInterval(() => {  
  //   setmydate(mydate => new Date());  
  //   // mydate = new Date();  
  // }, 1000);  
}, []);
```

React Hook : **useEffect** 改變 state 狀態

- 兩個參數
 - 第一個參數是 Effect function
 - 第二個則是 dependency array
 - [name, mydate], None

```
useEffect(() => {  
  setInterval(() => {  
    setmydate(mydate => new Date());  
    // mydate = new Date();  
  }, 1000);  
});
```



```
HTML CSS JS (Babel)
1 <div id="root">
2 </div>
3
4
5
6
7
8
9
10
11
12
13
14
15
const { useState } = React;
const Counter = () => {
  const [count, setCount] = useState(256)
  const increase = () => { setCount(count - 1) };
  const decrease = () => { setCount(count + 1) };

  return (
    <div className="container">
      <button className="chevron chevron-up" onClick={decrease}/>
      <span className="number" >{count}</span>
      <button className="chevron chevron-down" onClick={increase} />
    </div>
  )
};
ReactDOM.render(<Counter />, document.getElementById('root'));
```

252



```
1 const { useState, useEffect } = React;
2 const App = () => {
3   const [title, setTitle] = useState('posts')
4   const [lists, setLists] = useState([])
5   const changeTitle = (title) => () => {
6     setTitle(title)
7   }
8   useEffect(() => {
9     fetch(`https://jsonplaceholder.typicode.com/${title}`)
10      .then(res => res.json())
11      .then(json => setLists(json))
12    }, [title])
13    // console.log(json);
14    return (
15      <div className="container">
16        <div className="btn-group">
17          <div className="btn" onClick={changeTitle('posts')}>Post</div>
18          <div className="btn" onClick={changeTitle('users')}>Users</div>
19          <div className="btn" onClick={changeTitle('comments')}>Comments</div>
20        </div>
21        <section>
22          <h2>{title}</h2>
23          {lists.map(list => { return <pre>{JSON.stringify(list)}</pre> }}}
24        </section>
25      </div>
26    )
27  };
28 ReactDOM.render(<App />, document.getElementById('root'));
```

當 title 值改變時
就去呼叫 API

也就是把 title 的
內容掛上一個

React hook

title 值改變時，就去
做什麼事

POST

USERS

COMMENTS

POSTS

{"userId":1,"id":1,"title":"sunt aut facere repellat provident occaecati excepturi optio reprehenderit","body":

{"userId":1,"id":2,"title":"qui est esse","body":"est rerum tempore vitae\\nsequi sint nihil reprehenderit dolor



title改變 打API

```
1 const { useState, useEffect } = React;
2 const App = () => {
3   const [title, setTitle] = useState('posts')
4   const [lists, setLists] = useState([])
5   const changeTitle = (title) => () => {
6     setTitle(title)
7   }
8   useEffect(() => {
9     fetch(`https://jsonplaceholder.typicode.com/${title}`)
10      .then(res => res.json())
11      .then(json => setLists(json))
12    }, [title])
13    // console.log(json);
14    return (
15      <div className="container">
16        <div className="btn-group">
17          <div className="btn" onClick={changeTitle('posts')}>Post</div>
18          <div className="btn" onClick={changeTitle('users')}>Users</div>
19          <div className="btn" onClick={changeTitle('comments')}>Comments</div>
20        </div>
21        <section>
22          <h2>{title}</h2>
23          {lists.map(list => { return <pre>{JSON.stringify(list)}</pre> }}}
24        </section>
25      </div>
26    )
27  };
28  ReactDOM.render(<App />, document.getElementById('root'));
```

POST

USERS

COMMENTS

USERS

```
{"id":1,"name":"Leanne Graham","username":"Bret","email":"Sincere@april.biz","address":{"street":"Kulas Light",
{"id":2,"name":"Ervin Howell","username":"Antonette","email":"Shanna@melissa.tv","address":{"street":"Victor Pl
```

```
1 <div id="root"
2
3 </div>
```

```
1 const { useState, useEffect } = React;
2 const App = () => {
3   const [title, setTitle] = useState('posts')
4   const [lists, setLists] = useState([])
5   const changeTitle = (title) => () => {
6     setTitle(title)
7   }
8   useEffect(() => {
9     fetch(`https://jsonplaceholder.typicode.com/${title}`)
10      .then(res => res.json())
11      .then(json => setLists(json))
12   }, [title])
13   // console.log(json);
14   return (
15     <div className="container">
16       <div className="btn-group">
17         <div className="btn" onClick={changeTitle('posts')}>Post</div>
18         <div className="btn" onClick={changeTitle('users')}>Users</div>
19         <div className="btn" onClick={changeTitle('comments')}>Comments</div>
20       </div>
21       <section>
22         <h2>{title}</h2>
23         {lists.map(list => { return <pre>{JSON.stringify(list)}</pre> }}}
24       </section>
25     </div>
26   )
27 };
28 ReactDOM.render(<App />, document.getElementById('root'));
```

POST

USERS

COMMENTS

COMMENTS



title改變 打API

{"postId":1,"id":1,"name":"id labore ex et quam laborum","email":"Eliseo@gardner.biz","body":"laudantium enim q

{"postId":1,"id":2,"name":"quo vero reiciendis velit similique earum","email":"Jayne_Kuhic@sydney.com","body":"



時鐘例子：state寫法 用 **Class**

```
<script>
class Clock extends React.Component {
  constructor(props) {
    super(props);
    this.state = {date: new Date()};
  }
  componentDidMount() {
    this.interval = setInterval(() => this.tick(), 1000);
  }
  componentWillUnmount() {
    clearInterval(this.interval);
  }
  render() {
    return (
      <div>
        <h1>長庚標準時間 # state</h1>
        <h2>目前為： {this.state.date.toLocaleTimeString()}.</h2>
      </div>
    );
  }
  tick() {
    this.setState({
      date: new Date()
    });
  }
}
ReactDOM.render( <Clock />, document.getElementById('root'));
</script>
```

有點複雜
一個一個看

```
66 class Clock extends React.Component {
67   constructor(props) {
68     super(props);
69     this.state = {date: new Date()};
70   }
71   componentDidMount() {
72     this.interval = setInterval(() => this.tick(), 1000);
73   }
74   componentWillUnmount() {
75     clearInterval(this.interval);
76   }
77   render() {
78     return (
79       <div>
80         <h1>長庚標準時間 # state by class component</h1>
81         <h2>目前為: {this.state.date.toLocaleTimeString()}.</h2>
82       </div>
83     );
84   }
85   tick() {
86     this.setState({
87       date: new Date()
88     });
89   }
90 }
91 ReactDOM.render( <Clock />, document.getElementById('root'));
```



長庚標準時間 # state by class component

目前為： 上午10:42:24.

時鐘例子：state寫法

```
<script>
class Clock extends React.Component {
  constructor(props) {
    super(props);
    this.state = {date: new Date()};
  }
  componentDidMount() {
    this.interval = setInterval(() => this.tick(), 1000);
  }
  componentWillUnmount() {
    clearInterval(this.interval);
  }
  render() {
    return (
      <div>
        <h1>長庚標準時間 #state</h1>
        <h2>目前為： {this.state.date.toLocaleTimeString()}.</h2>
      </div>
    );
  }
  tick() {
    this.setState({
      date: new Date()
    });
  }
}
```



```
ReactDOM.render( <Clock />, document.getElementById('root'));
```

```
</script>
```

這裡一樣



<script>

```
class Clock extends React.Component {
```

```
  constructor(props) { // 初始 這個 component 時要跑的  
  }
```

```
  componentDidMount() { // 掛載 這個 component 時要跑的  
  }
```

```
  componentWillUnmount() { // 移除 這個 component 時要跑的  
  }
```

```
  render() { // render component (一定要有)  
  }
```

```
  tick() { // component private function (計算一秒一秒)  
  }
```

```
}
```

</script>

React Components Class 生命週期方法

- componentDidMount()
 - 在 component 被 render 到 DOM 之後才會執行
 - 只會被呼叫一次
- componentWillUnmount()
 - 從實體DOM階段移除「之前」的時候觸發
 - 只會被呼叫一次



建構者

state: 和render有關

```
<script>
class Clock extends React.Component {
  constructor(props) {
    super(props);
    this.state = {date: new Date()};
  }
  componentDidMount() {
    this.interval = setInterval(() => this.tick(), 1000);
  }
  componentWillUnmount() {
    clearInterval(this.interval);
  }
  render() {
    return (
      <div>
        <h1>長庚標準時間 # state by class component</h1>
        <h2>目前為: {this.state.date.toLocaleTimeString()}.</h2>
      </div>
    );
  }
  tick() {
    this.setState({
      date: new Date()
    });
  }
}
ReactDOM.render( <Clock />, document.getElementById('root'));
</script>
```



```
<script>
```

```
class Clock extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {date: new Date()};  
  }  
  componentDidMount() {  
    this.interval = setInterval(() => this.tick(), 1000);  
  }  
  componentWillUnmount() {  
    clearInterval(this.interval);  
  }  
}
```

```
  render() {  
    return (  
      <div>  
        <h1>長庚標準時間 # state by class component</h1>  
        <h2>目前為: {this.state.date.toLocaleTimeString()}</h2>  
      </div>  
    );  
  }  
  tick() {  
    this.setState({  
      date: new Date()  
    });  
  }  
}
```

```
ReactDOM.render( <Clock />, document.getElementById('root'));
```

```
</script>
```

元件初始/結束



```
<script>
```

```
class Clock extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {date: new Date()};  
  }  
  componentDidMount() {  
    this.interval = setInterval(() => this.tick(), 1000);  
  }  
  componentWillUnmount() {  
    clearInterval(this.interval);  
  }  
}
```

```
  render() {  
    return (  
      <div>  
        <h1>長庚標準時間 # state by class component</h1>  
        <h2>目前為: {this.state.date.toLocaleTimeString()}</h2>  
      </div>  
    );  
  }  
}
```

```
  tick() {  
    this.setState({  
      date: new Date()  
    });  
  }  
}
```

```
ReactDOM.render( <Clock />, document.getElementById('root'));
```

```
</script>
```

元件真正呈現



```
<script>
class Clock extends React.Component {
  constructor(props) {
    super(props);
    this.state = {date: new Date()};
  }
  componentDidMount() {
    this.interval = setInterval(() => this.tick(), 1000);
  }
  componentWillUnmount() {
    clearInterval(this.interval);
  }
  render() {
    return (
      <div>
        <h1>長庚標準時間 # state by class component</h1>
        <h2>目前為： {this.state.date.toLocaleTimeString()}.</h2>
      </div>
    );
  }
  tick() {
    this.setState({
      date: new Date()
    });
  }
}

ReactDOM.render( <Clock />, document.getElementById('root'));
</script>
```

Tick func 設定新的時間

```
66 class Clock extends React.Component {
67   constructor(props) {
68     super(props);
69     this.state = {date: new Date()};
70   }
71   componentDidMount() {
72     this.interval = setInterval(() => this.tick(), 1000);
73   }
74   componentWillUnmount() {
75     clearInterval(this.interval);
76   }
77   render() {
78     return (
79       <div>
80         <h1>長庚標準時間 # state by class component</h1>
81         <h2>目前為: {this.state.date.toLocaleTimeString()}.</h2>
82       </div>
83     );
84   }
85   tick() {
86     this.setState({
87       date: new Date()
88     });
89   }
90 }
91 ReactDOM.render( <Clock />, document.getElementById('root'));
```



長庚標準時間 # state by class component

目前為： 上午10:42:24.



HTML



CSS



JS (Babel)

```
1 <div id="root">
2   </div>
3
```

```
1 class Clock extends React.Component {
2   constructor(props) {
3     super(props);
4     this.state = {date: new Date()};
5     this.tick = this.tick.bind(this);
6   }
```

長庚標準時間 # state

Asia/Taipei 目前為： 上午11:45:34.

Europe/London 目前為： 上午4:45:34.

America/New_York 目前為： 下午11:45:33.



```
<script>
```

```
class Clock extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {date: new Date()};  
    this.tick = this.tick.bind(this);  
  }  
  componentDidMount() {  
    this.interval = setInterval(this.tick, 1000);  
  }  
  componentWillUnmount() {  
    clearInterval(this.interval);  
  }  
  render() {  
    return (  
      <div>  
        <h2>{this.props.timeZone} 目前為: {this.state.date.toLocaleTimeString('zh-TW', {timeZone:  
this.props.timeZone})}.</h2>  
        </div>  
      );  
    )  
  }  
  tick() {  
    this.setState({  
      date: new Date()  
    });  
  }  
}
```

```
ReactDOM.render(<div> <h1>長庚標準時間 # state</h1>  
  <Clock timeZone='Asia/Taipei' />  
  <Clock timeZone='Europe/London' />  
  <Clock timeZone='America/New_York' /></div>,  
  document.getElementById('root'));
```

```
</script>
```



使用 React Class 新增 DOM 元件

執行效果

```
7   };  
8   this.handleChange = this.handleChange.bind(this);  
9   this.handleSubmit = this.handleSubmit.bind(this);  
10  }  
11  handleChange(event) {  
12    this.setState({value: event.target.value});  
13  }  
14  handleSubmit(event) {  
15    this.state.itemList.push(this.state.value);
```

Name: ←

- cgu
- csie
- good ←



```
<script>
class NameForm extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      value: '',
      itemList: [],
    };
    this.handleChange = this.handleChange.bind(this);
    this.handleSubmit = this.handleSubmit.bind(this);
  }
  handleChange(event) {
    this.setState({value: event.target.value});
  }
  handleSubmit(event) {
    this.state.itemList.push(this.state.value);
    this.setState({value: '', itemList: this.state.itemList,});
    event.preventDefault();
  }
  render() {
    return (
      <div>
        <form onSubmit={this.handleSubmit}>
          <label>
            Name:
            <input type="text" value={this.state.value} onChange={this.handleChange} />
          </label>
          <input type="submit" value="Submit" />
        </form>
        <ul style={{ textAlign: 'left' }}>
          {this.state.itemList.map((item, index) => <li key={`item_${index}`}>{item}</li> ) }
        </ul>
      </div>
    );
  }
}
ReactDOM.render(<NameForm />,document.getElementById('root'));
</script>
```



使用 React Class 修改 MultiButton 元件

```
<script>
const changeText=(event)=>{
  console.log(event.target)
  event.target.innerText = event.target.innerText + "被點了"
}

const multiButton=(num)=>{
  var output=[];
  for(let i=1;i<num+1;++i)
    output.push(<button onClick={changeText}>第{i}個按鍵</button>)
  return output;
}

function App() {
  return (
    <div className="App">
      { multiButton(10) }
    </div>
  );
}
</script>
```

原本 multiButton 寫法

React Class multiButton 寫法



Hello World in React

chiou + Follow

HTML
CSS

JS (Babel)

```
1 class CounterButton extends React.Component {  
2   constructor(props) {  
3     super(props)  
4     this.state = {  
5       counter: 0  
6     }  
7     this.handleClick = this.handleClick.bind(this)  
8   }  
9   handleClick(event) {  
10    this.setState({ counter: this.state.counter+1 });  
11    console.log(this.state);  
12  }  
13  render() {  
14    var text = this.state.counter;  
15    return (  
16      <p onClick={this.handleClick}>  
17        你點了 {text} 次.  
18      </p>  
19    );  
20  }  
21 }
```



你點了 3 次.

現在我們用 React 寫 一個OX遊戲

		X
	O	
X		

下一位玩家：O

1. 遊戲開始
2. 回到 #1 步
3. 回到 #2 步
4. 回到 #3 步



```
body {
  font: 14px "Century Gothic", Futura, sans-serif;
  margin: 20px;
}

ol, ul {
  padding-left: 30px;
}

.board-row:after {
  clear: both;
  content: "";
  display: table;
}

.status {
  margin-bottom: 10px;
}

.square {
  background: #fff;
  border: 1px solid #999;
  float: left;
  font-size: 24px;
  font-weight: bold;
  line-height: 34px;
  height: 34px;
  margin-right: -1px;
  margin-top: -1px;
  padding: 0;
  text-align: center;
  width: 34px;
}

.square:focus {
  outline: none;
}

.kbd-navigation .square:focus {
  background: #ddd;
}

.game {
  display: flex;
  flex-direction: row;
}

.game-info {
  margin-left: 20px;
}
```

OX遊戲的css



OX遊戲的 index.js

```
1 ▶ function Square(props) {↔}
8
9 ▶ class Board extends React.Component {↔}
41
42 ▶ class Game extends React.Component {↔}
121
122  const root = ReactDOM.createRoot(document.getElementById("root"));
123  root.render(<Game />);
124
125 ▶ function calculateWinner(squares) {↔}
144
```

← React 進入點 **Game 元件**



Game 元件

```
42 ▶ class Game extends React.Component {  
43 ▶   constructor(props) {↔} ←  
55  
56 ▶   handleClick(i) {↔}  
74  
75 ▶   jumpTo(step) {↔}  
81  
82 ▶   render() {↔} ←  
120 }
```



Game 元件 constructor

```
43  constructor(props) {  
44      super(props);  
45      this.state = {  
46          history: [  
47              {  
48                  squares: Array(9).fill(null)  
49              }  
50          ],  
51          stepNumber: 0,  
52          xIsNext: true  
53      };  
54  }
```

← Game 元件紀錄每一步的狀態



每一次 render 檢查遊戲狀態

```
82 render() {
83   const history = this.state.history;
84   const current = history[this.state.stepNumber];
85   const winner = calculateWinner(current.squares);
86
87   const moves = history.map((step, move) => {
88     const desc = move ?
89       '回到 #' + move + ' 步':
90       '遊戲開始';
91     return (
92       <li key={move}>
93         <button onClick={() => this.jumpTo(move)}>{desc}</button>
94       </li>
95     );
96   });
97
98   let status;
99   if (winner) {
100     status = "贏家是 : " + winner;
101   } else {
102     status = "下一位玩家 : " + (this.state.xIsNext ? "X" : "O");
103   }
104
105   return (
106     <div className="game">
107       <div className="game-board">
108         <Board
109           squares={current.squares}
110           onClick={i => this.handleClick(i)}
111         />
112       </div>
113       <div className="game-info">
114         <div>{status}</div>
115         <ol>{moves}</ol>
116       </div>
117     </div>
118   );
119 }
```

Moves 元件 return JSX

Game 元件 return JSX

Board 元件

Moves 元件

Game 元件Render



1. 每一次 render 檢查遊戲狀態

```
82 render() {  
83   const history = this.state.history;  
84   const current = history[this.state.stepNumber];  
85   const winner = calculateWinner(current.squares);  
86  
87   const moves = history.map((step, move) => {  
88     const desc = move ?  
89       '回到 #' + move + ' 步':  
90       '遊戲開始';  
91     return (  
92       <li key={move}>  
93         <button onClick={() => this.jumpTo(move)}>{desc}</button>  
94       </li>  
95     );  
96   });  
97  
98   let status;  
99   if (winner) {  
100     status = "贏家是 : " + winner;  
101   } else {  
102     status = "下一位玩家 : " + (this.state.xIsNext ? "X" : "O");  
103   }  
104  
105   return (  
106     <div className="game">  
107       <div className="game-board">  
108         <Board  
109           squares={current.squares}  
110           onClick={i => this.handleClick(i)}  
111         />  
112       </div>  
113       <div className="game-info">  
114         <div>{status}</div>  
115         <ol>{moves}</ol>  
116       </div>  
117     </div>  
118   );  
119 }
```

3. jumpTo function

2. 處理滑鼠點擊事件

Game 元件Render



1. 每一次 render 檢查遊戲狀態

```
125 function calculateWinner(squares) {  
126   const lines = [  
127     [0, 1, 2],  
128     [3, 4, 5],  
129     [6, 7, 8],  
130     [0, 3, 6],  
131     [1, 4, 7],  
132     [2, 5, 8],  
133     [0, 4, 8],  
134     [2, 4, 6]  
135   ];  
136   for (let i = 0; i < lines.length; i++) {  
137     const [a, b, c] = lines[i];  
138     if (squares[a] && squares[a] === squares[b] && squares[a] === squares[c]) {  
139       return squares[a];  
140     }  
141   }  
142   return null;  
143 }
```

Game 元件Render

2. 處理滑鼠點擊事件

```
56 ▾ handleClick(i) { ←
57     const history = this.state.history.slice(0, this.state.stepNumber + 1);
58     const current = history[history.length - 1];
59     const squares = current.squares.slice();
60 ▾     if (calculateWinner(squares) || squares[i]) {
61         return;
62     }
63     squares[i] = this.state.xIsNext ? "X" : "O";
64 ▾     this.setState({
65 ▾         history: history.concat([
66 ▾             {
67                 squares: squares
68             }
69         ]),
70         stepNumber: history.length,
71         xIsNext: !this.state.xIsNext
72     });
73 }
```

Game 元件Render



```
75 ▾ | jumpTo(step) { ← 3. jumpTo function
76 ▾   this.setState({
77     stepNumber: step,
78     xIsNext: (step % 2) === 0
79   });
80   }
```

Game 元件Render



1. 每一次 render 檢查遊戲狀態

```
125 function calculateWinner(squares) {  
126   const lines = [  
127     [0, 1, 2],  
128     [3, 4, 5],  
129     [6, 7, 8],  
130     [0, 3, 6],  
131     [1, 4, 7],  
132     [2, 5, 8],  
133     [0, 4, 8],  
134     [2, 4, 6]  
135   ];  
136   for (let i = 0; i < lines.length; i++) {  
137     const [a, b, c] = lines[i];  
138     if (squares[a] && squares[a] === squares[b] && squares[a] === squares[c]) {  
139       return squares[a];  
140     }  
141   }  
142   return null;  
143 }
```

Game 元件Render



```
82 render() {
83   const history = this.state.history;
84   const current = history[this.state.stepNumber];
85   const winner = calculateWinner(current.squares);
86
87   const moves = history.map((step, move) => {
88     const desc = move ?
89       '回到 #' + move + ' 步':
90       '遊戲開始';
91     return (
92       <li key={move}>
93         <button onClick={() => this.jumpTo(move)}>{desc}</button>
94       </li>
95     );
96   });
97
98   let status;
99   if (winner) {
100     status = "贏家是 : " + winner;
101   } else {
102     status = "下一位玩家 : " + (this.state.xIsNext ? "X" : "O");
103   }
104
105   return (
106     <div className="game">
107       <div className="game-board">
108         <Board
109           squares={current.squares}
110           onClick={i => this.handleClick(i)}
111         />
112       </div>
113       <div className="game-info">
114         <div>{status}</div>
115         <ol>{moves}</ol>
116       </div>
117     </div>
118   );
119 }
```

Moves 元件 return JSX

Game 元件 return JSX

Board 元件

Moves 元件

Game 元件Render



```
9 class Board extends React.Component {  
10   renderSquare(i) {  
11     return (  
12       <Square  
13         value={this.props.squares[i]}  
14         onClick={() => this.props.onClick(i)}  
15       />  
16     );  
17   }  
18 }
```

← Square 元件

```
19 render() {  
20   return (← Board 元件 return JSX  
21     <div>  
22       <div className="board-row">  
23         {this.renderSquare(0)}  
24         {this.renderSquare(1)}  
25         {this.renderSquare(2)}  
26       </div>  
27       <div className="board-row">  
28         {this.renderSquare(3)}  
29         {this.renderSquare(4)}  
30         {this.renderSquare(5)}  
31       </div>  
32       <div className="board-row">  
33         {this.renderSquare(6)}  
34         {this.renderSquare(7)}  
35         {this.renderSquare(8)}  
36       </div>  
37     </div>  
38   );  
39 }  
40 }
```

Board 元件Render



```
1 function Square(props) { ← Square 元件
2   return ( ← Square 元件 return JSX
3     <button className="square" onClick={props.onClick}>
4       {props.value}
5     </button>
6   );
7 }
```

Square 元件Render

React 的 OX遊戲

O	X	O
X	O	X
O	X	

贏家是：O

1. 遊戲開始
2. 回到 #1 步
3. 回到 #2 步
4. 回到 #3 步
5. 回到 #4 步
6. 回到 #5 步
7. 回到 #6 步
8. 回到 #7 步
9. 回到 #8 步



ex#13: OX遊戲實作

時間到 16:00

- 請參考 page 32 - page 46
- 使用React Class 實作OX遊戲



ex#13: OX遊戲實作

時間到 16:00

- 請把整個 react project 上傳
 - 專案名：**react_game**
 - 除了 **build & node_modules** 這兩個目錄
 - 請使用 **gitignore**
- Commit to your GitHub (ex13 目錄)
 - **ex13/react_game**
 - **ex13** 裡面有一個 **react_game** 資料夾



Thanks!

Open for any questions

CJ Wu

cjwu@mail.cgu.edu.tw